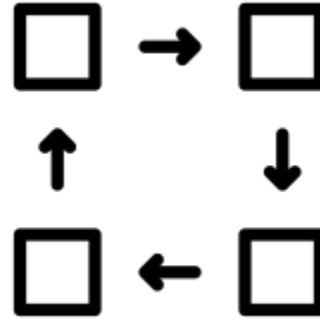




24-04-2026 10:30

Iteratori

Accedere in modo indiretto ad una sequenza di valori

Lezioni 15+ 16-
Corso 1



Iteratori

un'iteratore è un metodo che applico ad una collezione di dati che stabilisce se ho terminato di scandirla. Se non ha terminato, fornisce il prossimo elemento con un tipo `Option<T>` quindi ogni iteratore utilizza il metodo `next()` che restituisce un `Option<T>` qualora l'elemento esista

- Un iteratore è una **struttura dati** dotata di **stato**, in grado di generare una **sequenza** di valori
 - Esempio più semplice di iteratore:

```
for i in 0..10 { } // i è un iteratore che si ricorda dove è arrivato
```
 - I valori possono essere estratti da un **contenitore**, di cui l'iteratore detiene un riferimento, o generati programmaticamente, come nel caso di un intervallo di valori
- Un iteratore offre tipicamente un modo per verificare se sono presenti ulteriori valori da generare (`hasNext()` $\rightarrow$ `bool`) ed un altro per accedere al valore successivo (`next()` $\rightarrow$ `E`)
 - In Rust le due operazioni sono combinate (`next()` $\rightarrow$ `Option<T>`), in questo modo l'iteratore va avanti finché c'è un prossimo valore
 - l'iteratore restituisce `Some(T)` oppure `None`.
- Un iteratore può offrire ulteriori metodi che consentono di derivare un nuovo iteratore
 - Che trasforma la sequenza di valori in un'altra sequenza (accorpendo, eliminando, trasformando, ..., i singoli elementi originali)
- Pressoché tutti i linguaggi “moderni” offrono il concetto di iteratore come parte della propria libreria standard
 - Essi permettono di accedere ai valori contenuti all'interno di collezioni come liste, insiemi, mappe, scorrere i caratteri presenti all'interno di una stringa o leggere il contenuto di un file di testo estraendo una riga alla volta

Uso degli iteratori

- L'uso degli iteratori si sovrappone concettualmente a quello dei cicli for/while
 - Ma offre svariati vantaggi in termini di compattezza del codice, leggibilità, manutenibilità ed efficienza, nascondendo i dettagli necessari a generare / accedere ai singoli elementi
 - I vantaggi sono maggiormente evidenti se l'operazione svolta all'interno del ciclo è complessa e richiede, ad esempio, di scartare alcuni valori e di trasformare i restanti

ha saltato la spiegazione di questi due codici

```
let v1 = vec![1,2,3,4,5,6];  
let mut v2 = Vec::<String>::new();  
  
for i in 0..v1.len() {  
    if v1[i] %2 != 0 { continue; }  
    v2.push(format!("a{}",v1[i]))  
}  
println!("{:?}", v2); // ["a2","a4","a6"]
```

for.rs

```
let v1 = vec![1,2,3,4,5,6];  
let v2: Vec<String>;  
  
v2 = v1.iter()  
    .filter(|val| { *val %2 == 0 })  
    .map(|val| format!("a{}",val))  
    .collect();  
  
println!("{:?}", v2); // ["a2","a4","a6"]
```

iter0.rs

gli iteratori operano in modo pigro (lazy) —> IMPORTANTE l'azione fatta dall'iteratore si manifesta solo su richiesta di qualcun altro —> la chiamata dell'iteratore non produce un risultato

Caratteristiche degli iteratori

Fornire tutti i dati in una volta è sconveniente sia in termini di memoria sia perchè posso non aver bisogno di certi dati. Invece, Rust in questo è lazy —> restituisce un solo elemento e solo on demand.

- Un iteratore offre un modo uniforme di accedere agli elementi, indipendentemente da come essi siano generati o da dove siano prelevati
 - Permettendo al codice che utilizza tali valori di ignorare la fonte (ed il tipo di collezione) ed essere più generico e nascondendo i dettagli dell'operazione
- Gli iteratori operano in modalità pigra (**lazy**)
 - Solo a fronte della richiesta di un valore successivo, si occupano di generarlo / prelevarlo
 - Le operazioni fatte dagli iteratori non sono fatte in anticipo, ma solo 1 alla volta e su richiesta
 - Efficiente per lavorare con collezioni molto grandi
- Gli iteratori possono abilitare l'elaborazione parallela dei dati ospitati in una collezione
 - Permettendo di sfruttare la presenza di più core nell'elaboratore
- La possibilità di derivare un iteratore da un altro iteratore aumenta la flessibilità del codice
 - E' facile inserire all'interno di una catena di elaborazione passi ulteriori che iniettano nuovi valori, ne eliminano altri, combinano più valori tra loro, modificano l'ordine di visita, ..., senza dover intervenire sul codice che utilizza i valori generati

Fonte dei dati

- Gli iteratori operano su una fonte di dati che può essere:
 - un contenitore di dati (un vettore, un array, una lista, una mappa, un file, ecc.) che contiene già i dati e chiediamo di esplorarli
 - un generatore di numeri casuali o un range di valori che genera un dato ogni volta che lo chiediamo.

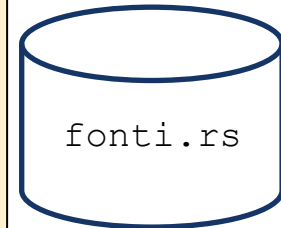
range implementa il tratto iterator, quindi utilizza il metodo next che restituisce l'elemento successivo di tipo Item

```
use rand::Rng;

fn main() {
    let v1: Vec<i32> = (1..10).collect();
    let numeri: Vec<u32> = (0..10)
        .map(|_| rand::thread_rng().gen_range(1..=100))
        .collect();

    println!("{:?} \n{:?}", v1, numeri);
}
```

collect() è un consumatore che produce un risultato, in questo caso un vettore che quindi contiene il risultato



Iteratori in Rust

come trasformiamo una struttura dati in un iterabile?
introduciamo due tratti

1) `Iterator` : ha un metodo molto importante, `next()`, che è necessario per restituire un elemento di una collezione

- In Rust, un **iteratore** è una qualsiasi struttura dati che implementa il tratto **`std::iter::Iterator`**

```
trait Iterator{  
    type Item;  
    fn next(&mut self) -> Option<Self::Item>;  
    ...// molti altri metodi con implementazione di default  
}
```

- **`Item`**: tipo del valore che l'iteratore produce
- **`next()`**: ritorna un `Option` (se c'è ritorna `Some(Self::Item)`, altrimenti ritorna `None`).
- **`&mut self`**: dal momento che lavora su sè stesso modificando lo stato.

Iteratori e possesso

- I **contenitori** presenti nella libreria standard mettono normalmente a disposizione tre metodi per ricavare un iteratore ai dati contenuti al loro interno
 - **iter()**, crea un iteratore che restituisce oggetti di tipo **&Item** e non consuma il contenuto del contenitore
 - **iter_mut()**, crea un iteratore che restituisce oggetti di tipo **&mut Item** e permette di modificare gli elementi all'interno del contenitore
 - **into_iter()**, crea un iteratore che prende possesso del contenitore (e lo consuma) e restituisce oggetti di tipo **Item** estraendoli dal contenitore.

```
fn main() {
    let numbers = [1, 2, 3, 4, 5];

    // Itera attraverso ciascun elemento dell'array e stampalo

    for num in numbers.iter() {
        println!("{}", num);
    }

    for num in &numbers {
        println!("{}", num);
    }
}
```

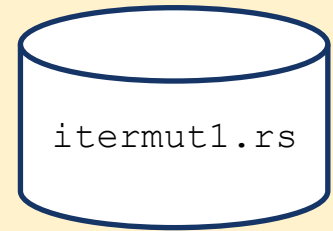
il ciclo for richiede al .iter() di fornirgli un elemento alla volta UTILIZZANDO, CHIAMANDO IL METODO NEXT.
Questo significa che implicitamente viene chiamato il metodo next

l'iter crea l'oggetto che permette di scandire numbers

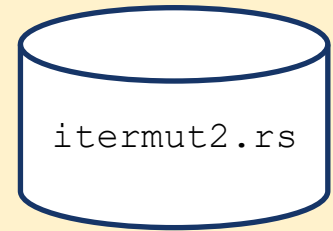


i due cicli for sono equivalenti


```
fn main() {  
    let mut numbers = vec![1, 2, 3, 4, 5];  
  
    // Incrementa di 1 ogni numero nel vettore utilizzando iter_mut()  
    for num in numbers.iter_mut() {  
        *num += 1;  
    }  
  
    // Stampa il vettore dopo l'incremento  
    println!("{:?}", numbers); // Output: [2, 3, 4, 5, 6]  
  
    // Incrementa ogni numero nel vettore di 1 utilizzando &mut  
    for num in &mut numbers {  
        *num += 1;  
    }  
  
    // Stampa il vettore dopo l'incremento  
    println!("{:?}", numbers); // Output: [3, 4, 5, 6, 7]  
}
```



```
fn main() {  
    let mut v = vec![String::from("a"), String::from("b"), String::from("c")];  
  
    for s in &mut v {  
        s.push_str("1"); // s: &mut String - Modifico il contenuto del vettore  
    }  
  
    for s in v.iter_mut() {  
        s.push_str("bis");  
    }  
  
    for s in &v {  
        println!("{}", s); // Output: a1bis, b1bis, c1bis  
    }  
}
```



```
fn main() {
```

```
    let v = vec![10.0, 30.0, 50.0, 90.0];
```

```
    let mut sum=f64::default();
```

```
    // into_iter consuma il contenitore
```

```
    for num in v.into_iter() {
```

```
        sum += num;
```

```
    }
```

```
    println!("{}", sum);
```

```
    // equivalente a
```

```
    // let sum: i32 = v.into_iter().sum();
```

```
    let v2 = vec![1, 3, 5, 9];
```

```
    let mut sum2=i32::default();
```

```
    for num2 in v2 {
```

```
        sum2 += num2;
```

```
    }
```

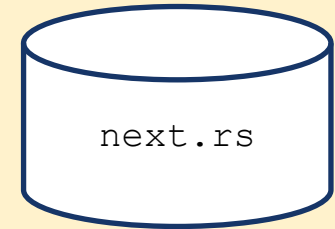
```
    println!("{}", sum2);
```

```
}
```

se uso `into_iter()`, quando il `for` finisce, il vettore non è più accessibile



```
fn main() {  
    let mut numbers = vec![1, 2, 3, 4, 5];  
  
    // Ottiene il primo elemento dall'iteratore  
    if let Some(first_num) = numbers.iter().next() {  
        println!("Il primo numero e' {}", first_num);  
    } else {  
        println!("Nessun elemento trovato");  
    }  
  
    if let Some(first_num) = numbers.iter_mut().next() {  
        *first_num += 1;  
    } else {  
        println!("Nessun elemento trovato");  
    }  
  
    for num in numbers.iter() {  
        println!("{}", num);  
    }  
}
```



```
// Definiamo una struttura che implementerà il tratto Iterator
```

```
struct Contatore {  
    count: usize,  
    max: usize,  
}
```

```
// Implementiamo il tratto Iterator
```

```
impl Iterator for Contatore {  
    type Item = usize;  
    fn next(&mut self) -> Option<Self::Item> {  
        if self.count < self.max {  
            let val = self.count;  
            self.count += 1;  
            Some(val)  
        } else {  
            None  
        }  
    }  
}
```

```
}  
fn main() {  
    let mut contatore = Contatore { count: 0, max: 10 };  
  
    // Usiamo l'iteratore per stampare i valori  
    while let Some(i) = contatore.next() {  
        println!("{}", i);  
    }  
}
```

```
trait Iterator{  
    type Item;  
    fn next(&mut self) -> Option<Self::Item>;  
    ...  
}
```

Text



```

struct MyRange {
    count: usize,
}

impl Iterator for MyRange {
    type Item = usize;

    fn next(&mut self) -> Option<Self::Item> {
        if self.count == 0 {
            None
        } else {
            self.count -= 1;
            Some(self.count)
        }
    }
}

```



rendo la variabile iterabile implementando Iterator alla struttura dati.
Il ciclo for chiama implicitamente il metodo next

```

fn main() {
    let range_iter = MyRange {count: 20};

    for n in range_iter {
        println!("Next number: {}", n);
    }
}

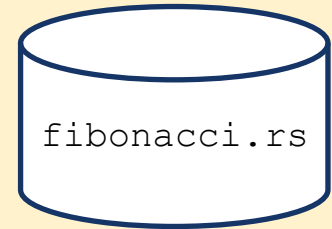
```

chiedi se il tipo ritornato da next dipende in questo caso dal tipo del campo count della struct

```
struct ContatoreFibonacci {
    a: usize,
    b: usize,
    max: usize,
}

impl Iterator for ContatoreFibonacci {
    type Item = usize;
    // Definiamo il metodo next per ottenere il prossimo elemento
    fn next(&mut self) -> Option<Self::Item> {
        if self.a < self.max {
            let valore = self.a;
            let nuovo_valore = self.a + self.b;
            self.a = self.b;
            self.b = nuovo_valore;
            Some(valore)
        } else {
            None
        }
    }
}

fn main() {
    let fibonacci_iteratore = ContatoreFibonacci { a: 0, b: 1, max:1000 };
    for numero in fibonacci_iteratore {
        println!("Fibonacci: {}", numero);
    }
}
```

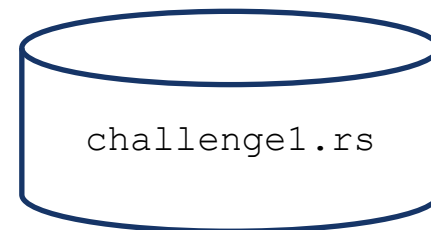




```
fn main() {  
  
    let v = vec![1,2,3,4,5,6,7,8,9,10];  
    for num in v { // equivale a: for num in v.into_iter()  
        if num % 3 == 0  
        { break; }  
    }  
  
}
```



creo un iteratore che consuma la collezione
dopo l'esecuzione dell'istruzione break il vettore viene consumato nonostante
non è stato fatto il processamento degli altri elementi



Che cosa succede al vettore dopo il break?

Rispondete sul canale Discord Rust

<https://discord.gg/CZdtKDwjU4>

l'iteratore non viene più utilizzato dopo il break,
ed esegue il drop su tutti gli altri elementi della
collezione

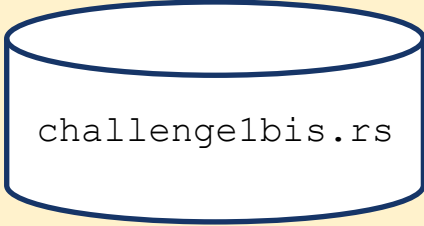
Per capire il comportamento eseguire questo

```
#[derive(Debug)]
struct S (i32);

impl Drop for S {
    fn drop(&mut self) {
        println!("Dropping S ({})", self.0);
    }
}

fn main() {
    let v = vec![S(1),S(2),S(3),S(4),S(5),S(6),S(7),S(8),S(9),S(10)];

    for num in v {
        println!("{:?}",num);
        if num.0 % 3 == 0
            { break; }
    }
}
```



challenge1bis.rs

chiedi —> quando viene eseguito break non ho capito come fa l'iteratore a fare il drop se deve essere chiamato next

Tipo Iterable

- Un tipo **iterable** può segnalare la capacità di essere esplorato tramite un iteratore, implementando il tratto **std::iter::IntoIterator**
- Il tratto **IntoIterator** è un tratto di conversione che consente a un tipo di essere convertito in un iteratore
- Per implementare **IntoIterator** per un tipo personalizzato, è necessario definire un metodo **into_iter** che restituisca un iteratore per la collezione. Questo iteratore deve implementare il tratto **Iterator** e fornire un metodo **next** che restituisca gli elementi della collezione uno alla volta.

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {  
    type Item;  
    type IntoIter: Iterator;  
    fn into_iter(self) -> Self::IntoIter;  
}
```

Esempio

Si vuole poter iterare sui campi di una struct

```
struct Coppia {  
    a: i32,  
    b: i32,  
}
```

obiettivo: creare un iteratore per scandire i valori di questa Struct.
Per usare un ciclo for occorre che la collezione su cui facciamo la scansione sia iterabile. Per questo motivo occorre convertire la struct in un iterabile

per scandire i dati della struct devo prima di tutto rendere la struttura iterabile

Primo passo: creare un Iteratore

- Occorre definire un tipo iteratore e implementare il tratto `Iterator`. L'iteratore deve mantenere uno stato interno per tenere traccia della posizione corrente nell'iterazione.

costruire un iteratore significa implementare il metodo `next`, che permette di scandire l'iteratore e fornire l'elemento successivo.

`next` restituisce un `option`, ovvero dichiara se esiste un elemento successivo oppure se la collezione è stata scansionata completamente

con `underscore`, soddisfiamo l'esaustività del pattern matching

```
struct CoppiaIter {  
    stato: u8,  
    a: i32,  
    b: i32,  
}  
  
impl Iterator for CoppiaIter {  
    type Item = i32;  
    fn next(&mut self) -> Option<Self::Item> {  
        match self.stato {  
            0 => {  
                self.stato = 1;  
                Some(self.a)  
            }  
            1 => {  
                self.stato = 2;  
                Some(self.b)  
            }  
            _ => None,  
        }  
    }  
}
```

se `b` avesse un tipo diverso come imposto il valore di `Item`?
non è possibile implementare il tratto `Iterator` per la struct

Secondo passo: trasformare il dato in un iteratore

- Occorre definire il tratto `IntoIterator` per convertire la struct originale nell'iteratore.

11:39 28-04-2026

da 11:41 spiega il programma
`coppiaiter2.rs` che trovi sul portale



quando andiamo a costruire le varie
implementazione dei metodi
next bisogna fornire la versione
adeguata per fare o il possesso (e
quindi consumare
il dato) oppure il riferimento

```
impl IntoIterator for Coppia {  
    type Item = i32;  
    type IntoIter = CoppiaIter;  
  
    fn into_iter(self) -> Self::IntoIter {  
        CoppiaIter {  
            stato: 0,  
            a: self.a,  
            b: self.b,  
        }  
    }  
}  
  
fn main() {  
    let c = Coppia { a: 10, b: 20 };  
    for val in c.into_iter() {  
        println!("{}", val);  
    }  
}
```



```
struct Pixel {
    r: i8,
    g: i8,
    b: i8,
}
```

```
impl IntoIterator for Pixel {
```

```
    type Item = i8;
```

```
    type IntoIter = std::array::IntoIter<i8, 3>;
```

```
    fn into_iter(self) -> Self::IntoIter {
```

```
        [self.r, self.g, self.b].into_iter()
```

```
    }
```

```
} MORALE -> into_iter applicato al pixel coincide con l'applicare into_iter applicato ad un array
```

```
fn main() {
```

```
    let pixel = Pixel { r: 54, g: 23, b: 74 };
```

```
    let mut iter = pixel.into_iter();
```

```
    // Chiamare next restituirà il prossimo elemento dell'iteratore, se presente
```

```
    if let Some(component) = iter.next() {
```

```
        println!("Il primo componente è: {}", component);
```

```
    }
```

```
    if let Some(component) = iter.next() { // Puoi continuare a chiamare next per ottenere i successivi elementi
```

```
        println!("Il secondo componente è: {}", component);
```

```
    }
```

```
    if let Some(component) = iter.next() {
```

```
        println!("Il terzo componente è: {}", component);
```

```
    }
```

```
    if let Some(_component) = iter.next() { // dopo che tutti gli elementi sono stati consumati: None
```

```
        println!("Questo non verrà stampato perché non ci sono più elementi.");
```

```
    } else {
```

```
        println!("Non ci sono più componenti.");
```

```
    }
```

```
    let pixel2 = Pixel { r: 5, g: 2, b: 7 };
```

```
    for p in pixel2.into_iter() {
```

```
        println!("{:?}", p);
```

```
    }
```

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {
    type Item;
    type IntoIter: Iterator;
    fn into_iter(self) -> Self::IntoIter;
}
```

ci appoggiamo ad un tipo iterabile. Per rendere

iterabile la struct Pixel implementiamo IntoIterator e quindi implementiamo into_iter,

ma APPLICHIAMO INTO_ITER AD UN TIPO ITERABILE

intoiterator1.rs

la variabile iter è un iteratore

INTO_ITER SERVE PER RENDERE UN TIPO ITERABILE

posso chiamare anche il ciclo for sulla struct Pixel, perchè ho reso la struct iterabile

```
struct Libro {
    titolo: String,
    pagine: Vec<String>,
}
```

il vettore di stringhe è iterabile, perchè un vettore è iterabile

su cosa itero? —> qui trasformo la struttura libro è un iteratore e definisco che voglio iterare sulle pagine

```
// Implementiamo IntoIterator per la struttura Libro
```

```
impl IntoIterator for Libro {
```

```
    type Item = String; // Il tipo degli elementi che l'iteratore produrrà
```

```
    type IntoIter = std::vec::IntoIter<String>; // Il tipo dell'iteratore
```

```
    fn into_iter(self) -> Self::IntoIter {
```

```
        // self.pagine è un Vec<String>, e into_iter() su Vec
```

```
        // restituisce un iteratore che consuma il Vec.
```

```
        self.pagine.into_iter()
```

```
    }
```

```
}
```

```
fn main() {
```

```
    let libro = Libro { // Creiamo un nuovo libro per dimostrare l'iterazione esplicita
```

```
        titolo: String::from("Alice nel Paese delle Meraviglie"),
```

```
        pagine: vec![ String::from("Prologo"), String::from("Capitolo 1: Giù nella tana del coniglio")],
```

```
    };
```

```
    let mut iteratore_pagine = libro.into_iter();
```

```
    while let Some(pagina) = iteratore_pagine.next() {
```

```
        println!("Pagina: {}", pagina);
```

```
    }
```

```
}
```

intoiterator3.rs

11:50 commenta il codice intoiterator4.rs
che trovi sul portale, RIASCOLTA

Terminologia



- **Iterator:** qualunque tipo che implementa il Tratto Iterator. I tipi iterator devono definire il metodo `next` che restituisce i valori successivi nella sequenza
- **Iterable:** qualunque tipo che implementa il Tratto Intolterator
- Un **iterator** è ricavato da un tipo **iterable** chiamando il metodo `into_iter()`
- Un iterator produce **values**
- Il singolo valore che l'iteratore produce si chiama **item**
- Il codice che riceve gli items che un iteratore produce e genera un nuovo iteratore è detto **adapter**
- Il codice che consuma un iteratore e produce un risultato è detto **consumer**.

il vettore è un classico esempio di struttura dati che può essere scandita. Quindi mi serve un iteratore che sia in grado di processare i dati, ovvero che scandisca gli elementi fornendo un next()

Adattatori

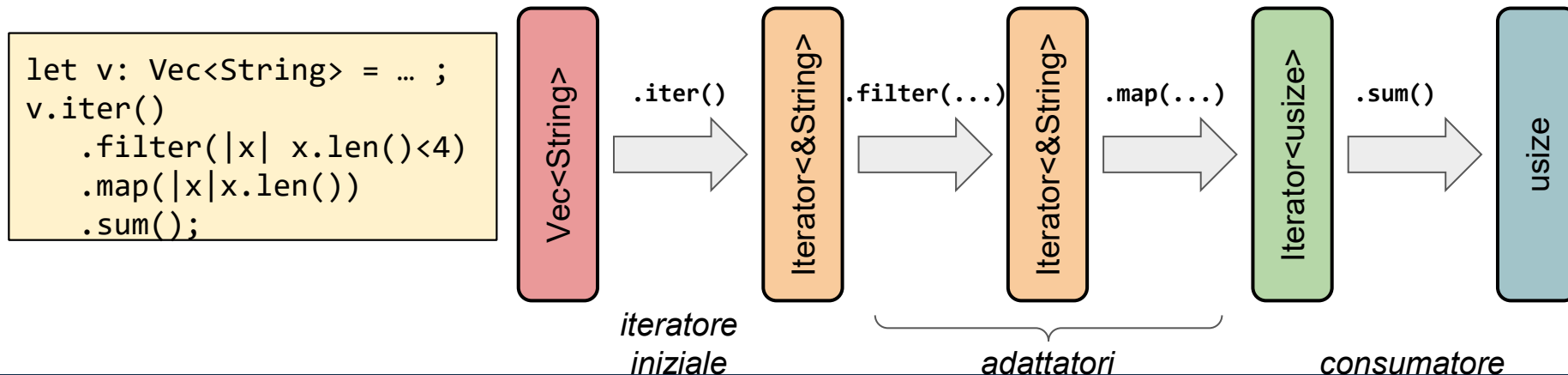
quando chiamo `v.iter()` NON STO PRODUCENDO UN RISULTATO

l'adattore, ovvero `.filter()`, `.map()` etc prende un iteratore, ovvero un oggetto in grado di scandire una sequenza, e producono come risultato un iteratore

- Il tratto `Iterator` definisce un nutrito gruppo di metodi che consumano un iteratore e ne derivano uno differente, in grado di offrire funzionalità ulteriori
 - Possono essere combinati in catene più o meno lunghe al termine delle quali occorre porre un consumatore finale
 - Tutti gli adattatori sono infatti pigri (lazy) di natura e non invocano il metodo `next()` dell'oggetto a monte se non a seguito di una richiesta proveniente da un loro consumatore
 - Catena reattiva: non fa nulla finché non viene messo un consumatore (terminale) che raccoglie i dati

il consumatore, ovvero l'ultimo metodo della catena, prende in input un iteratore

ragiona da sinistra verso destra per capire come evolvono i dati



11:54 28-04-2026 IMPORTANTE (sono riuscito a scrivere tutto qui sotto)

nella slide precedente costruiamo da un iteratore iniziale. Applicando il `.iter()` creo un iteratore.

Gli adattatori SONO LAZY. Il loro scopo è prendere un iteratore per generare un altro iteratore. Il

loro utilizzo corrisponde esattamente alla chiamata del metodo `next`, CHE AVVIENE SOLO SU RICHIESTA DI CIO'

che sta a valle. Infatti a valle di tutto CI VUOLE UN CONSUMATORE, ovvero un metodo particolare che produce

un risultato finale. Per poter fornire un risultato, il consumatore DEVE OTTENERE UN VALORE. In questo senso, l'adattatore che sta a monte deve chiamare il metodo `next`. L'iteratore chiama `next` fino a quando non ha finito: chiama ripetutamente il metodo `next` fino all'esaurimento della collezione.

Il modo migliore per ragionare e capire cosa sta facendo il codice è partire da sinistra verso destra, perchè io parto dai dati, a cui applico dei metodi. Tuttavia, operativamente il codice eseguibile ha un procedimento che si scatena da destra verso sinistra, ma alla fine il risultato è lo stesso.

partendo dal fondo capisco quello che scatena l'esecuzione, perchè tutti gli iteratori della catena sono lazy

eseguire l'iteratore vuol dire chiamare il metodo next, perchè fornisce un nuovo elemento della collezione

Adattatori

- **map<B, F>(self, f: F) -> Map<Self, F>**
 - Esegue la chiusura ricevuta come argomento su ogni elemento dell'iteratore ritornato
- **filter<P>(self, predicate: P) -> Filter<Self, P>**
 - Ritorna un iteratore che restituisce solo gli elementi per i quali l'esecuzione della chiusura ricevuta come argomento ritorna true
- **filter_map<B, F>(self, f: F) -> FilterMap<Self, F>**
 - Concatena in maniera concisa filter e map, l'iteratore risultante conterrà solo elementi per i quali la chiusura ritorna Some(B)
- **flatten(self) -> Flatten<Self>**
 - Ritorna un iteratore dal quale sono state rimosse le strutture annidate
 - `vec![vec![1,2,3,4],vec![5,6]].into_iter().flatten().collect::<Vec<u8>>()==&[1,2,3,4,5,6]`
- **flat_map<U, F>(self, f: F) -> FlatMap<Self, U, F>**
 - Concatena in maniera concisa map e flatten, esegue la chiusura ricevuta e rimuove le strutture annidate
- **take(self, n: usize) -> Take<Self>**
 - Ritorna un iteratore che contiene al più i primi n elementi dell'iteratore su cui viene eseguito (meno, se l'iteratore originale non contiene abbastanza elementi)

Adattatori

- **`take_while<P>(self, predicate: P) -> TakeWhile<Self, P>`**
 - Esegue la funzione ricevuta su tutti gli elementi dell'iteratore originale, conserva tutti gli elementi fino a quando la funzione ritorna true; dal momento in cui diventa false, scarta tutti i valori rimanenti
- **`skip(self, n: usize) -> Skip<Self>`**
 - Ritorna un iteratore che esclude i primi n elementi dell'iteratore su cui viene eseguito, se si raggiunge la fine ritorna un iteratore vuoto
- **`skip_while<P>(self, predicate: P) -> SkipWhile<Self, P>`**
 - Esegue la funzione ricevuta su tutti gli elementi dell'iteratore originale, esclude tutti gli elementi fino a quando la funzione ritorna false, dal momento in cui diventa true conserva tutti i valori rimanenti
- **`peekable(self) -> Peekable<Self>`**
 - Ritorna un iteratore sul quale è possibile chiamare i metodi `peek()` e `peek_mut()` per accedere al valore successivo senza consumarlo.
- **`fuse(self) -> Fuse<Self>`**
 - Ritorna un iteratore che termina dopo il primo None
- **`rev(self) -> Rev<Self>`**
 - Ritorna un iteratore con la direzione invertita

Adattatori

- **inspect<F>(self, f: F) -> Inspect<Self, F>**
 - Ogni volta che riceve una richiesta, preleva un elemento dall'iteratore a monte e la passa sia alla funzione, che ha possibilità di ispezionarlo, che al consumatore a valle
- **chain<U>(self, other: U) -> Chain<Self, <U as IntoIterator>::IntoIter>**
 - Prende come argomento un iteratore e lo concatena all'originale, ritorna un nuovo iteratore
- **enumerate(self) -> Enumerate<Self>**
 - Ritorna un iteratore che restituisce una tupla formata dall'indice dell'iterazione e dal valore (i,val)
- **zip<U>(self, other: U) -> Zip<Self, <U as IntoIterator>::IntoIter>**
 - Combina due iteratori per ritornare un nuovo iteratore che ha come elementi le coppie composte dai valori dei primi due iteratori
- **by_ref(&mut self) -> &mut Self**
 - Prende in prestito un iteratore senza consumarlo, lasciando intatto il possesso dell'originale
- **copied<'a, T>(self) -> Copied<Self>**
 - Ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale vengono **copiati**
- **cloned<'a, T>(self) -> Cloned<Self>**
 - Ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale vengono **clonati**
- **cycle(self) -> Cycle<Self>**
 - Raggiunta la fine di un iteratore riparte dall'inizio, ciclando all'infinito
- ...

map()

12:00 28-04-2026

- Trasforma gli elementi di un iteratore applicando una funzione a ciascun elemento e restituendo un nuovo iteratore che contiene i risultati della trasformazione.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Creiamo un iteratore dal vettore
    // applichiamo il metodo map per raddoppiare ogni elemento

    let doubled_iter = numbers.iter().map(|&x| x * 2);

    // Stampiamo i risultati
    for doubled in doubled_iter {
        println!("{}", doubled);
    }
}
```

notiamo che se non viene usato il ciclo for, di per se .iter() non fa nulla e in questo è lazy

.map() è un metodo molto importante che applica una funzione ad ogni elemento dell'iterabile

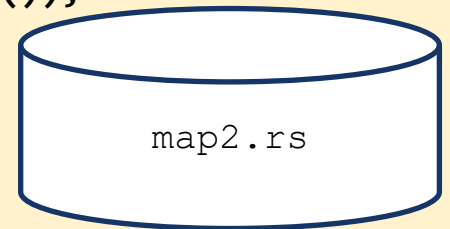
con .iter() creo un iteratore

anche double_iter è un iteratore. Il ciclo for scandisce l'iteratore e mi fornisce i valori



```
fn main() {  
    let words = vec!["hello", "world", "how", "are", "you"];  
  
    // Creiamo un iteratore dal vettore  
    // applichiamo il metodo map per ottenere la lunghezza di ogni parola  
  
    let word_uppercase_iter = words.iter().map(|word| word.len());  
  
    // Stampiamo i risultati  
    for name in word_uppercase_iter {  
        println!("{}", name);  
    }  
  
    // applichiamo il metodo map per ottenere la conversione di ogni parola  
    let word_uppercase_iter = words.iter().map(|w| w.to_uppercase());  
  
    // Stampiamo i risultati  
    for name in word_uppercase_iter {  
        println!("{}", name);  
    }  
}
```

chiedi perchè qui viene usato il ref e nel codice del filter non viene usato



filter()

12:06

- Crea un nuovo iteratore che include solo gli elementi che soddisfano un predicato.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
  
    // Utilizzo di filtri per selezionare solo i numeri pari  
    let even_numbers = numbers.iter().filter(|&x| x % 2 == 0);  
  
    // Stampiamo i numeri pari  
    for n in even_numbers {  
        println!("Even number: {}", n);  
    }  
}
```



```
fn main() {  
    let nomi = vec!["Alice", "Bob", "Anna", "Carl", "David"];  
    let nomi_filtrati = nomi.iter()  
        .filter(|&nome| nome.starts_with("A"));  
  
    for n in nomi_filtrati {  
        println!("{:?}", n); // Stampa: ["Alice", "Anna"]  
    }  
}
```



filter2.rs

filter_map()

- Applica una funzione a ciascun elemento e restituisce solo gli elementi risultanti che non sono None.

```
fn main() {  
    let dati = vec!["42", "93", "NaN", "18", "77", "invalido"];  
    let numeri_validi = dati.into_iter()  
        .filter_map(|s| s.parse::<i32>().ok());  
  
    for n in numeri_validi {  
        println!("{:?}", n);  
    } // Stampa: [42, 93, 18, 77]  
}
```

viene applicata la funzione di parsificazione. il filtro è dato da .ok() → perchè se option è None il valore viene filtrato e quindi non viene incluso nel vettore numeri_validi

filtermap1.rs

take()

- Crea un nuovo iteratore che restituisce solo un numero specificato di elementi iniziali. Una volta raggiunto il limite specificato, l'iteratore smette di restituire elementi.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Prende solo i primi 3 numeri dall'inizio del vettore  
    let first_three = numbers.iter().take(3);  
  
    for n in first_three {  
        println!("{:?}", n);  
    } // Output: [1, 2, 3]  
}
```



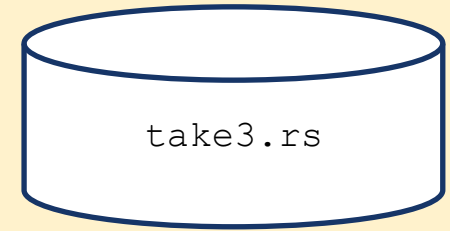
codice importante

- 1) parti dal vettore e seleziona quelli maggiori di venti → `.filter(|&num| *num > 20)`
- 2) prendi i primi due → `.take(2)`

```
fn main() {  
    let numbers = vec![10, 20, 30, 40, 50];  
  
    // Prende i primi due numeri maggiori di 20 dall'inizio del vettore  
    let first_two_over_twenty = numbers.iter()  
        .filter(|&num| *num > 20)  
        .take(2);  
    for n in first_two_over_twenty {  
        println!("Primi due numeri maggiori di 20: {:?}", n);  
    }  
}
```



```
fn main() {  
    let numbers = vec![10, 20, 30, 40, 50];  
  
    let first_two_over_twenty = numbers.iter()  
        .take(3)  
        .filter(|&num| *num > 10);  
  
    for n in first_two_over_twenty {  
        println!("Numeri maggiori di 10 tra i primi 3: {:?}", n);  
    }  
}
```



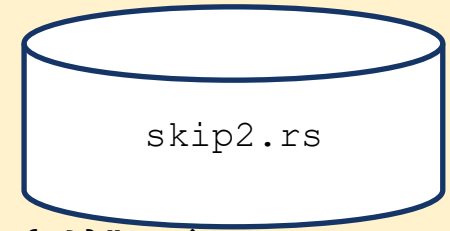
skip()

- Crea un nuovo iteratore che salta un numero specificato di elementi all'inizio dell'iteratore originale e restituisce tutti gli elementi rimanenti.

```
fn main() {  
    let numbers = vec![1, 4, 5, 6, 7];  
  
    // Salta i primi due numeri  
    let skipped = numbers.iter().skip(2);  
  
    for n in skipped {  
        println!("Valori dopo i primi 2: {:?}", n);  
    }  
}
```



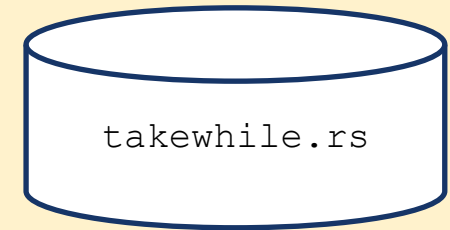
```
fn main() {  
    let numbers = vec![1, 40, 5, 60, 7, 45, 34, 99];  
  
    // Salta i primi due numeri  
    let skipped = numbers.iter()  
        .skip(2)  
        .filter(|&num| *num > 10)  
        .take(3);  
  
    for n in skipped {  
        println!("Dopo i primi 2, i primi 3 valori superiori a 10: {:?})", n);  
    }  
}
```



take_while()

- Crea un nuovo iteratore che restituisce gli elementi finché una determinata condizione è vera. Una volta che la condizione diventa falsa, l'iteratore smette di restituire elementi.

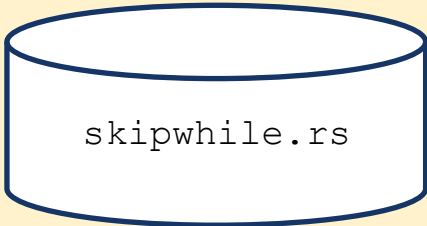
```
fn main() {  
    let numbers = vec![5, 10, 15, 20, 22, 30];  
  
    // Prende numeri finché sono multipli di 5  
    let multiples_of_five = numbers.iter()  
        .take_while(|&num| *num % 5 == 0);  
  
    for n in multiples_of_five {  
        println!("Multipli di 5: {:?}", n);  
    }  
}
```



skip_while()

- Crea un nuovo iteratore che salta elementi fino a quando un predicato fornisce true. Una volta che il predicato restituisce false per la prima volta, tutti gli elementi successivi (compreso il primo per cui il predicato è diventato false) vengono inclusi nell'iteratore risultante.

```
fn main() {  
    let numbers = vec![1, 3, 5, 7, 2, 7, 8, 9, 10];  
  
    // Salta numeri fino a quando non trova un numero pari  
    let skipped = numbers.iter()  
        .skip_while(|&num| *num % 2 != 0);  
  
    for n in skipped {  
        println!("Tutti i numeri a partire dal primo pari: {:?}", n);  
    }  
}
```

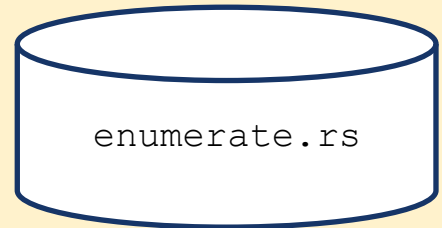


skipwhile.rs

enumerate()

- Genera un iteratore che produce la tupla (indice, valore) per ciascun elemento di una sequenza

```
fn main() {  
    let my_vec = vec!["a", "b", "c"];  
  
    // Otteniamo un iteratore che produce coppie (indice, valore)  
    let mut iter = my_vec.iter().enumerate();  
  
    // Iteriamo attraverso ogni coppia (indice, valore)  
    while let Some((index, value)) = iter.next() {  
        println!("Indice: {}, Valore: {}", index, value);  
    }  
}
```



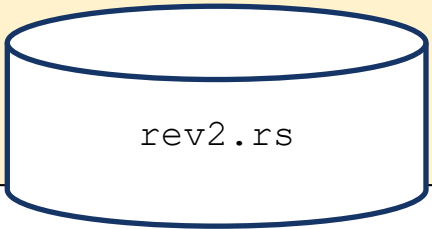
rev()

- Utilizzato per invertire l'ordine degli elementi.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
    let reversed_numbers= numbers.iter().rev();  
  
    println!("Numeri originali: {:?}", numbers);  
  
    for n in reversed_numbers {  
        println!("Numeri invertiti: {:?}", n);  
    }  
}
```



```
fn main() {  
    // Creazione di un range da 1 a 5  
    let range = 1..=5;  
  
    // Inversione dell'ordine del range utilizzando il metodo rev  
    let reversed_range= range.rev();  
  
    for n in reversed_range {  
        println!("Nuovo Range: {:?}", n);  
    }  
}
```



rev2.rs

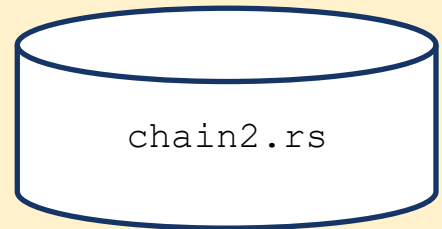
chain()

- Appende un iteratore ad un altro

```
fn main() {  
    let numbers1 = vec![1, 2, 3];  
    let numbers2 = vec![4, 5, 6, 7];  
  
    // Concateniamo i due vettori utilizzando il metodo chain  
    let chained_numbers = numbers1  
        .iter()  
        .chain(numbers2.iter());  
    for n in chained_numbers {  
        println!("Numeri concatenati: {:?}", n);  
    }  
}
```



```
fn main() {  
    let strings = vec!["hello", "world"];  
    let numbers = vec![1, 2, 3];  
  
    // Concateniamo il vettore di stringhe con il vettore di numeri  
    let concatenated = strings.iter()  
        .map(|s| s.to_string())  
        .chain(numbers.iter().map(|&n| n.to_string()));  
  
    for s in concatenated {  
        println!("elemento: {:?}", s);  
    }  
}
```



flatten()

- Appiattisce una struttura dati composta da iterabili annidati e genera un singolo iteratore che contiene tutti gli elementi.

```
fn main() {  
    let nested_numbers = vec![vec![1, 2, 3], vec![4, 5, 6], vec![7, 8, 9]];
  
    // Appiattisce la struttura nested di vettori in un singolo vettore  
    let flattened_numbers = nested_numbers.into_iter().flatten();
  
    for n in flattened_numbers {  
        println!("Numeri appiattiti: {:?}", n);  
    }  
}
```



flat_map()

- Consente di mappare ogni elemento dell'iteratore in un altro iteratore, quindi appiattisce il risultato in un singolo iteratore.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4];  
    let new_numbers = numbers.iter()  
        .flat_map(|&x| vec![x, x * x, x * x * x]);  
  
    for n in new_numbers {  
        println!("{:?}", n);  
    } // Output: [1, 1, 1, 2, 4, 8, 3, 9, 27, 4, 16, 64]  
}
```



peekable()

- Crea un iteratore che permette di guardare l'elemento successivo senza che l'iteratore proceda al prossimo elemento
- I metodi `peek()` e `peek_mut()` permettono di accedere al valore successivo senza consumarlo.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
    let mut peekable_numbers = numbers.iter().peekable();  
    // Controlla se c'è un elemento successivo e lo stampa senza consumarlo  
    if let Some(&next_number) = peekable_numbers.peek() {  
        println!("Elemento successivo: {}", next_number);  
    }  
    // Ora si può consumare l'elemento  
    if let Some(next_number) = peekable_numbers.next() {  
        println!("Elemento consumato: {}", next_number);  
    }  
    for number in peekable_numbers {  
        println!("Elemento successivo: {}", number); // Stampa gli elementi rimanenti  
    }  
}
```



```
fn main() {  
    let numeri = vec![1, 2, 3, 4];  
    let mut iteratore_peekable = numeri.iter().peekable();  
  
    while let Some(numero) = iteratore_peekable.next() {  
        println!("Valore dall'iteratore: {}", numero);  
        match iteratore_peekable.peek() {  
            Some(prossimo) => println!("Sbirciando il prossimo valore: {}", prossimo),  
            None => println!("Non ci sono altri valori"),  
        }  
    }  
}
```

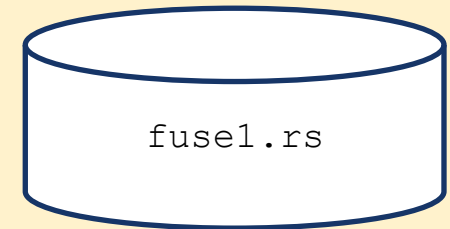


peekable2.rs

fuse()

- Crea un iteratore che si ferma al primo None che incontra e continuerà a restituire None.

```
fn main() {  
    let numbers = vec![1, 2, 3];  
    let mut iter = numbers.iter().fuse();  
  
    // Stampa i primi due numeri  
    println!("First: {:?}", iter.next()); // Some(1)  
    println!("Second: {:?}", iter.next()); // Some(2)  
    println!("Third: {:?}", iter.next()); // Some(3)  
  
    // Stampa None e Continua a restituire None  
    println!("Fourth: {:?}", iter.next()); // None  
    println!("Fifth: {:?}", iter.next()); // None  
}
```



```
fn main() {  
    let numbers = vec![1, 2, 3];  
    let mut iter = numbers.iter().fuse();  
  
    // Consuma l'iteratore completamente  
    println!("Iteratore 1:");  
    while let Some(&number) = iter.next() {  
        println!("Numero: {}", number);  
    }  
  
    // Riprova a consumare l'iteratore  
    println!("Iteratore 2:");  
    while let Some(&number) = iter.next() {  
        println!("Numero: {}", number);  
    }  
}
```



inspect()

- Utilizzato per scopi di debug: applica una chiusura all'item senza modificare il valore

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Utilizziamo inspect per stampare ogni elemento prima di moltiplicarlo per 2  
    let doubled_numbers = numbers.iter()  
        .inspect(|&x| println!("Elemento: {}", x))  
        .map(|&x| x * 2);  
  
    for n in doubled_numbers {  
        println!("Raddoppiato: {:?}", n);  
    }  
}
```



```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Utilizziamo inspect per controllare se un elemento è pari o dispari  
    let parity_check= numbers.iter()  
        .inspect(|&x| {  
            if x % 2 == 0 {  
                println!("{}", x);  
            } else {  
                println!("{}", x);  
            }  
        });  
  
    for n in parity_check {  
        println!("Numero: {:?}", n);  
    }  
}
```



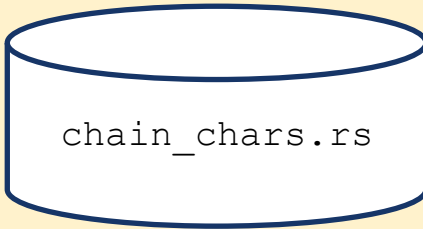
inspect2.rs

chars()

adattatore importante

- Chiamato su una stringa restituisce un iteratore che produce sequenzialmente i caratteri all'interno della stringa

```
fn main() {  
    let words = vec!["hello", "world"];  
    let chars = "123";  
  
    // Concateniamo l'iteratore delle parole con l'iteratore dei caratteri  
    let chained_sequence = words.iter()  
        .flat_map(|word| word.chars())  
        .chain(chars.chars());  
  
    for n in chained_sequence{  
        println!("Carattere: {:?}", n);  
    }  
}
```

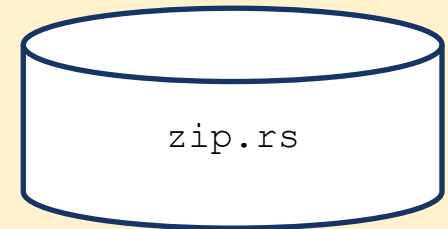


chain_chars.rs

zip()

- Combina 2 iteratori in un solo iteratore che produce una sequenza di tuple che contengono un elemento da ciascun iteratore.

```
fn main() {  
    let numbers = vec![1, 2, 3];  
    let letters = vec!['a', 'b', 'c'];  
  
    // Otteniamo un iteratore che produce tuple (numero, lettera)  
    let mut iter = numbers.iter().zip(&letters);  
  
    // Iteriamo attraverso ogni coppia (numero, lettera)  
    while let Some((number, letter)) = iter.next() {  
        println!("Numero: {}, Lettera: {}", number, letter);  
    }  
}
```



unzip()

- Scompatta una collezione di tuple in 2 collezioni separate, una contenente tutti gli elementi nella prima posizione delle tuple e l'altra contenente tutti gli elementi nella seconda posizione.

```
fn main() {  
    let data = vec![(1, 'a'), (2, 'b'), (3, 'c')];  
  
    // Unzip della collezione di tuple in due iteratori separati  
    let (numbers, characters): (Vec<_>, Vec<_>) = data.into_iter().unzip();  
  
    println!("Numeri: {:?}", numbers);  
    println!("Caratteri: {:?}", characters);  
}
```



unzip.rs

by_ref()

- Utilizzato per ottenere un riferimento all'iteratore corrente senza consumarlo
- Utile quando si desidera utilizzare l'iteratore più volte senza consumarlo o per evitare di dover possedere l'iteratore.

```
fn main()
{
    let mut numeri = vec![1, 2, 3].into_iter();

    // Utilizziamo by_ref per prendere i primi due elementi
    // senza consumare completamente l'iteratore
    let primi_due = numeri.by_ref().take(2);

    for n in primi_due {
        println!("Primi 2: {}", n);
    }
    for n in numeri {
        println!("Tutti i numeri: {}", n);
    }
}
```



cloned()

- Prende un iteratore che produce riferimenti e restituisce un iteratore che produce valori clonati da questi riferimenti

```
fn main() {  
    let words = vec!["hello", "world", "rust"];  
  
    // Otteniamo un iteratore che produce copie delle stringhe  
    let cloned_iter = words.iter().cloned();  
  
    // Stampiamo ogni parola nel nuovo iteratore  
    for word in cloned_iter {  
        println!("Parola: {}", word);  
    }  
    for word in words {  
        println!("Parola: {}", word);  
    }  
}
```



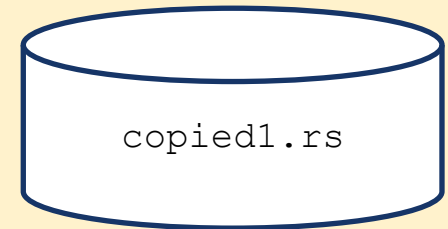
```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Crea un iteratore che restituisce valori clonati  
    let iter = numbers.iter().cloned();  
  
    // Filtra e mappa i valori clonati  
    let doubled_even_numbers = iter  
        .filter(|n| n % 2 == 0)  
        .map(|n| n * 2);  
  
    // Stampa i numeri pari raddoppiati  
    println!("Doubled even numbers:");  
    for n in doubled_even_numbers {  
        println!("{}", n);  
    }  
  
    // L'iteratore originale non è consumato, quindi possiamo utilizzarlo di nuovo  
    println!("Original numbers:");  
    for n in numbers {  
        println!("{}", n);  
    }  
}
```



copied()

- Genera un iteratore che produce una copia dei valori originali.

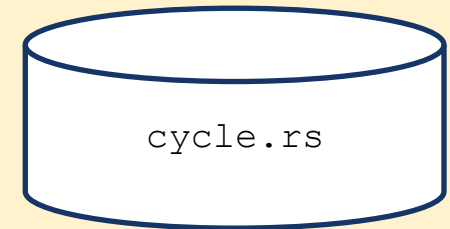
```
fn main() {  
    let tuple_vec = vec![(1, 'a'), (2, 'b'), (3, 'c')];  
  
    // Otteniamo un iteratore che produce copie delle tuple originali  
    let copied_iter = tuple_vec.iter().copied();  
  
    // Stampiamo ogni tupla nel nuovo iteratore  
    for tuple in copied_iter {  
        println!("Tuple: {:?}", tuple);  
    }  
    for tuple in tuple_vec {  
        println!("Tuple: {:?}", tuple);  
    }  
}
```



cycle()

- Genera un iteratore che può ciclare ripetutamente attraverso gli elementi di una sequenza

```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    // Otteniamo un iteratore che cicla ripetutamente attraverso i numeri  
    let mut cycle_iter = numbers.iter().cycle();  
  
    // Stampiamo i primi 5 numeri del ciclo  
    for _ in 0..5 {  
        if let Some(num) = cycle_iter.next() {  
            println!("Numero: {}", num);  
        }  
    }  
}
```



Consumatori

12:14 28-04-2026

parto da un iteratore e produco un risultato

- **collect(self) -> B**
 - Trasforma un iteratore in una collezione
- **for_each<F>(self, f: F)**
 - Esegue la chiusura ricevuta su tutti gli elementi dell'iteratore
- **try_for_each<F, R>(&mut self, f: F) -> R**
 - Esegue una chiusura che può fallire su tutti gli elementi dell'iteratore, si ferma dopo il primo fallimento
- **nth(&mut self, n: usize) -> Option<Self::Item>**
 - Ritorna l'ennesimo elemento dell'iteratore
- **all<F>(&mut self, f: F) -> bool**
 - Verifica che la chiusura ricevuta restituisca true per tutti gli elementi restituiti dall'iteratore
- **any<F>(&mut self, f: F) -> bool**
 - Verifica che la chiusura ricevuta restituisca true per almeno un elemento restituito dall'iteratore
- **find<P>(&mut self, predicate: P) -> Option<Self::Item>**
 - Cerca un elemento sulla base della chiusura ricevuta come argomento e lo ritorna
- **count(self) -> usize**
 - Ritorna il numero di elementi dell'iteratore
- **sum<S>(self) -> S**
 - Somma tutti gli elementi di un iteratore e ritorna il valore ottenuto

Consumatori

- **`product<P>(self) -> P`**
 - Moltiplica tutti gli elementi di un iteratore e ritorna il valore ottenuto
- **`max(self) -> Option<Self::Item>`**
 - Ritorna il massimo tra gli elementi dell'iteratore, se trova due massimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None
- **`max_by_key<B, F>(self, f: F) -> Option<Self::Item>`**
 - Esegue la chiusura ricevuta come argomento su tutti gli elementi e ritorna quello che produce il risultato massimo
- **`min(self) -> Option<Self::Item>`**
 - Ritorna il minimo tra gli elementi dell'iteratore, se trova due minimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None
- **`min_by_key<B, F>(self, f: F) -> Option<Self::Item>`**
 - Esegue la chiusura ricevuta come argomento su tutti gli elementi e ritorna quello che produce il risultato minimo

Consumatori

- **`position<P>(&mut self, predicate: P) -> Option<usize>`**
 - Cerca un elemento sulla base della chiusura ricevuta come argomento e ritorna la posizione
- **`rposition<P>(&mut self, predicate: P) -> Option<usize>`**
 - Cerca un elemento sulla base della chiusura ricevuta come argomento, partendo da destra e ritornando la posizione
- **`last(self) -> Option<Self::Item>`**
 - Ritorna l'ultimo elemento dell'iteratore
- **`fold<B, F>(self, init: B, f: F) -> B`**
 - Esegue la chiusura ricevuta accumulando i risultati sul primo argomento ricevuto
- **`try_fold<B, F, R>(&mut self, init: B, f: F) -> R`**
 - Esegue la chiusura ricevuta fino a quando ritorna con successo, accumulando i risultati sul primo argomento ricevuto
- **`find_map<B, F>(&mut self, f: F) -> Option<B>`**
 - Esegue la chiusura ricevuta su tutti gli elementi e ritorna il primo risultato valido
- **`partition<B, F>(self, f: F) -> (B, B)`**
 - Consuma un iteratore e ritorna due collezioni sulla base del predicato ricevuto
- **`reduce<F>(self, f: F) -> Option<Self::Item>`**
 - Riduce l'iteratore ad un singolo elemento eseguendo la funzione ricevuta

collect()

collect serve per ricostruire una struttura dati a partire da un iteratore

- Consuma un iteratore e crea una collezione.

nell'utilizzo di collect devo sempre specificare al compilatore il tipo; tipicamente usa un `Vec<_>`. In questo caso l'underscore dice al compilatore "il tipo degli elementi inferiscilo dal tipo della collezione che ti ho passato, ovvero numbers"

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
    // Filtriamo solo i numeri pari e li raccogliamo in un nuovo vettore  
    let even_numbers: Vec<_> = numbers.iter()  
        .filter(|&x| x % 2 == 0)  
        .collect();  
  
    println!("Numeri pari: {:?}", even_numbers);  
}
```

.collect() inserisce dentro il vettore il risultato del filtraggio



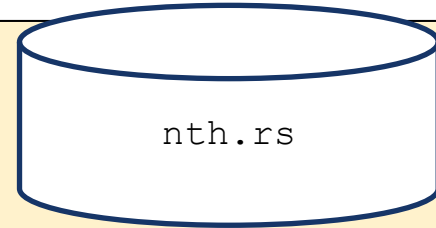
pensala così per capire nth() → dato un insieme di dati prendi il numero 2 per ottenere il risultato, ovvero solo il secondo elemento a partire dall'iteratore che sta a monte

nth()

- Utilizzato per ottenere l'elemento all'indice specificato.

```
fn main() {  
    let numbers = vec![10, 20, 30, 40, 50];  
  
    // Otteniamo il terzo elemento del vettore  
    match numbers.iter().nth(2) {  
        Some(&number) => println!("Terzo elemento: {}", number),  
        None => println!("Nessun elemento trovato all'indice specificato."),  
    }  
  
    // Otteniamo il secondo elemento del vettore  
    match numbers.iter().nth(1) {  
        Some(&number) => println!("Secondo elemento: {}", number),  
        None => println!("Nessun elemento trovato all'indice specificato."),  
    }  
}
```

poichè è stato usato un match, puoi capire
che il risultato ottenuto da .nth() è di tipo Option<T>



all()

- Verifica che tutti gli elementi di un iteratore soddisfano una determinata condizione.

```
fn main() {  
    let numbers = vec![2, 4, 6, 8, 10];  
  
    // Verifichiamo se tutti gli elementi sono pari  
    let all_even = numbers.iter().all(|&x| x % 2 == 0);  
  
    if all_even {  
        println!("Tutti gli elementi sono pari.");  
    } else {  
        println!("Almeno un elemento non è pari.");  
    }  
}
```



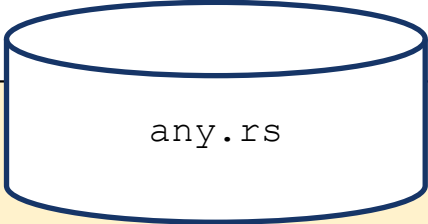
all.rs

any()

- Verifica se almeno un elemento soddisfa una condizione.

```
fn main() {  
    let words = vec!["hello", "world", "rust", "programming"];  
  
    // Verifichiamo se almeno una stringa ha una lunghezza maggiore di 5 caratteri  
    let any_long_word = words.iter().any(|&word| word.len() > 5);  
  
    if any_long_word {  
        println!("Almeno una parola ha una lunghezza maggiore di 5 caratteri.");  
    } else {  
        println!("Nessuna parola ha una lunghezza maggiore di 5 caratteri.");  
    }  
}
```

puoi capire che any produce un booleano perchè viene usata if any_long_word



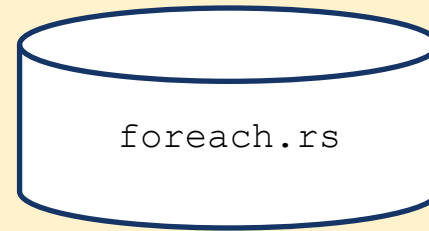
any.rs

for_each()

- Applica una chiusura su ciascun elemento

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Stampiamo ogni numero del vettore  
    numbers.iter().for_each(|&num| {  
        println!("Numero: {}", num);  
    });  
}
```

Text

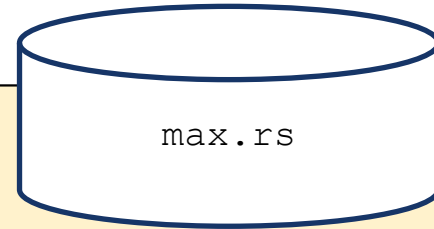


il fatto che chiamo il next per scandire tutti i valori dell'iteratore è implicito nel metodo `for_each()`

max()

- Applicato su un iteratore che contiene elementi confrontabili, restituisce un Option, che in caso di successo riporta il valore massimo.

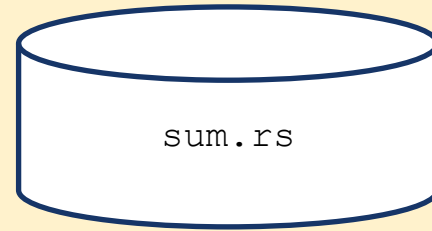
```
fn main() {  
    let numbers = vec![10, 30, 20, 50, 40];  
  
    // Trova il massimo elemento nell'iteratore  
    let max_number = numbers.iter().max();  
  
    match max_number {  
        Some(max) => println!("Il massimo elemento è: {}", max),  
        None => println!("L'iteratore è vuoto."),  
    }  
}
```



sum()

- Applicato su un iteratore calcola la somma di tutti gli elementi dell'iteratore.

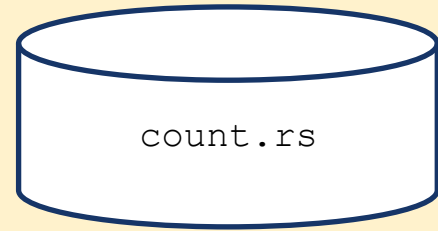
```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Calcola la somma di tutti gli elementi nell'iteratore  
    let sum: i32 = numbers.iter().sum();  
  
    println!("La somma di tutti gli elementi è: {}", sum);  
}
```



count()

- Applicato su un iteratore conta quanti elementi appartengono alla sequenza

```
fn main() {  
    let numbers = vec![2, 4, 6, 7, 8, 9];  
  
    let odd = numbers.iter().filter(|&x| *x % 2 != 0).count();  
  
    println!("Il numero di dispari è: {}", odd);  
}
```



find()

- Applicato su un iteratore cerca il primo elemento che soddisfa una determinata condizione. Restituisce `Some(elemento)` se l'elemento è stato trovato, altrimenti restituisce `None`.

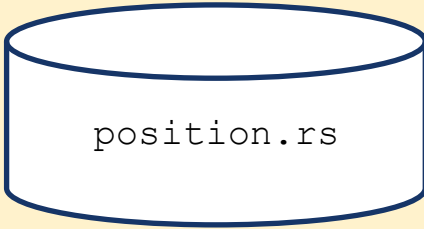
```
fn main() {  
    let numbers = vec![2, 4, 6, 7, 8, 9];  
  
    // Trova il primo numero dispari nell'iteratore  
    let first_odd = numbers.iter().find(|&x| *x % 2 != 0);  
  
    match first_odd {  
        Some(odd) => println!("Il primo numero dispari è: {}", odd),  
        None => println!("Nessun numero dispari trovato."),  
    }  
}
```



position()

- Metodo utilizzato per trovare l'indice del primo elemento che soddisfa un determinato predicato

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Find the index of the first even number  
    let index = numbers.iter().position(|&x| x % 2 == 0);  
  
    match index {  
        Some(i) => println!("The first even number is at index {}", i),  
        None => println!("No even number found"),  
    }  
}
```



position.rs

last()

- Metodo utilizzato per trovare l'ultimo elemento di una sequenza

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Ottieni l'ultimo numero pari  
    let last_even = numbers  
        .iter()  
        .filter(|&x| x % 2 == 0)  
        .last();  
  
    match last_even {  
        Some(&number) => println!("L'ultimo numero pari è: {}", number),  
        None => println!("Non ci sono numeri pari"),  
    }  
}
```



try_for_each()

- Applica una chiusura (che può fallire) su ciascun elemento

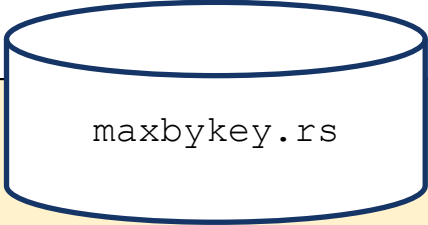
```
fn main() {  
    let strings = vec!["programming", "computer"];  
  
    // Stampa la lunghezza di ciascuna stringa, gestendo gli errori  
    let result = strings.iter().try_for_each(|s| -> Result<(), ()> {  
        if s.len() > 5 {  
            println!("La lunghezza di '{}' è maggiore di 5.", s);  
            Ok(())  
        } else {  
            Err(()) // Interrompe l'iterazione se la lunghezza è minore o uguale a 5  
        }  
    });  
    // Controlla il risultato dell'iterazione  
    match result {  
        Ok(()) => println!("Tutte le stringhe hanno una lunghezza maggiore di 5."),  
        Err(()) => println!("Almeno una stringa ha una lunghezza minore o uguale a 5."),  
    }  
}
```

tryforeach.rs

max_by_key()

- Applicato su un iteratore per calcolare il massimo di un iteratore in base al risultato di una funzione di confronto personalizzata.

```
fn main() {  
    let words = vec!["hello", "world", "rust", "programming"];  
  
    // Trova la stringa con il maggior numero di caratteri  
    let longest_word = words.iter().max_by_key(|word| word.len());  
  
    match longest_word {  
        Some(longest) => println!("La stringa più lunga è: {}", longest),  
        None => println!("L'iteratore è vuoto."),  
    }  
}
```



maxbykey.rs

product()

- Calcola il prodotto tra gli elementi di una sequenza.

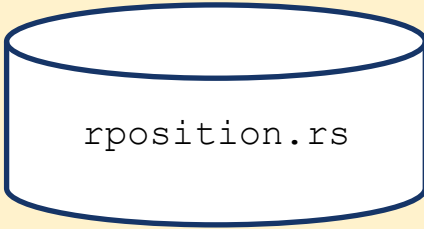
```
fn main() {  
    let numbers = vec![1, 2, 3];  
  
    // Calcola il prodotto di tutti gli elementi dell'iteratore  
    let product: u32 = numbers.iter().product();  
  
    println!("Il prodotto di {:?} è {}", numbers, product);  
}
```



rposition()

- Metodo utilizzato per trovare l'indice del primo elemento che soddisfa un determinato predicato partendo dal fondo

```
fn main() {  
    let numbers = vec![1, 3, 5];  
  
    // Find the index of the first even number  
    let index = numbers.iter().rposition(|&x| x % 2 == 0);  
  
    match index {  
        Some(i) => println!("The last even number is at index {}", i),  
        None => println!("No even number found"),  
    }  
}
```

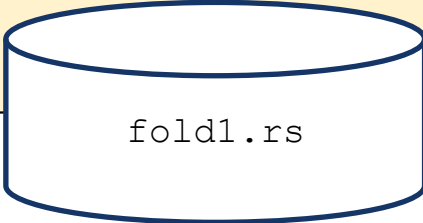


rposition.rs

fold()

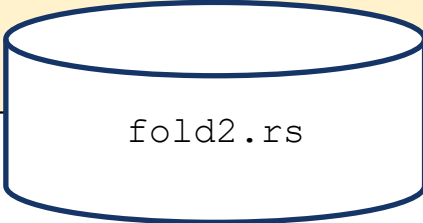
- Esegue la chiusura ricevuta accumulando i risultati sul primo argomento ricevuto

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Somma tutti gli elementi della collezione  
    let sum = numbers.iter().fold(0, |accumulatore, &x| accumulatore + x);  
  
    println!("La somma di tutti gli elementi è: {}", sum);  
}
```



fold1.rs

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Calcola il prodotto di tutti gli elementi utilizzando il metodo fold()  
    let product = numbers.iter().fold(1, |acc, n| acc * n);  
  
    println!("Il prodotto di {:?} è {}", numbers, product);  
}
```

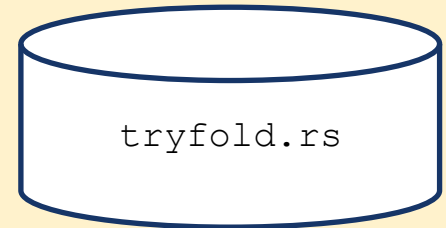


fold2.rs

try_fold()

- Simile a fold, ma gestisce anche gli errori.

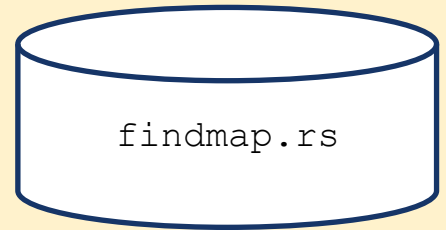
```
fn main() {  
    let numbers = vec![2, 3, 4, 5];  
  
    // Calcola il prodotto di tutti gli elementi della collezione  
    let product = numbers.iter().try_fold(1, |acc, &x| {  
        if x != 0 {  
            Ok(acc * x)  
        } else {  
            Err("Zero")  
        }  
    });  
  
    match product {  
        Ok(result) => println!("Il prodotto di tutti gli elementi è: {}", result),  
        Err(err) => println!("Errore: {}", err),  
    }  
}
```



find_map()

- Cerca il primo elemento che soddisfa un predicato e restituisce il risultato di una funzione di mapping applicata a quell'elemento.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Cerca il primo numero pari e raddoppia il valore  
    let result = numbers.iter().find_map(|&x| {  
        if x % 2 == 0 {  
            Some(x * 2)  
        } else {  
            None  
        }  
    });  
  
    match result {  
        Some(value) => println!("Il primo numero pari raddoppiato è: {}", value),  
        None => println!("Nessun numero pari trovato"),  
    }  
}
```



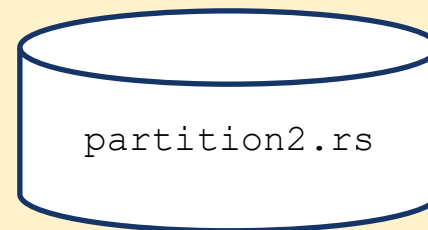
partition()

- Divide una collezione in due parti, in base a un predicato fornito
- Restituisce una tupla contenente gli elementi che soddisfano il predicato e l'altra quelli che non lo soddisfano.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    // Dividi la collezione in numeri pari e dispari  
    let (even_numbers, odd_numbers): (Vec<_>, Vec<_>) =  
        numbers  
        .into_iter()  
        .partition(|&x| x % 2 == 0);  
  
    println!("Numeri pari: {:?}", even_numbers);  
    println!("Numeri dispari: {:?}", odd_numbers);  
}
```



```
fn main() {  
    let people = vec![  
        ("Alice", 30),  
        ("Bob", 25),  
        ("Charlie", 35),  
        ("David", 40),  
    ];  
  
    // Dividi la collezione in due parti in base all'età  
    let (young, old): (Vec<_>,Vec<_>) = people.into_iter().partition(|&(_, age)| age < 35);  
  
    println!("Giovani: {:?}", young);  
    println!("Anziani: {:?}", old);  
}
```



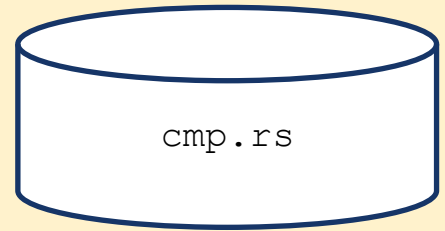
Confronti tra iteratori

- **`cmp<I>(self, other: I) -> Ordering`**
 - Confronta gli elementi di due iteratori
- **`eq<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di due iteratori sono uguali
- **`ne<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di due iteratori sono diversi
- **`lt<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di un iteratore sono minori rispetto a quelli di un secondo iteratore
- **`le<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di un iteratore sono minori o uguali rispetto a quelli di un secondo iteratore
- **`gt<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di un iteratore sono maggiori rispetto a quelli di un secondo iteratore
- **`ge<I>(self, other: I) -> bool`**
 - Verifica se gli elementi di un iteratore sono maggiori o uguali rispetto a quelli di un secondo iteratore
- ...

cmp()

- 2 iteratori possono essere confrontati convertendoli prima in collezioni di valori comparabili e poi confrontando le collezioni utilizzando il metodo cmp
- Il confronto avviene confrontando gli elementi corrispondenti delle 2 collezioni, considerando l'ordine degli elementi e il numero di elementi.

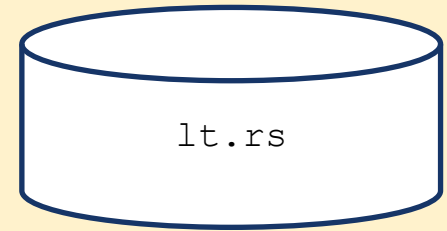
```
fn main() {  
    let numbers1 = vec![1, 3, 2];  
    let numbers2 = vec![1, 2, 4];  
  
    // Confronto dei due vettori  
    let comparison = numbers1.iter().cmp(numbers2.iter());  
  
    match comparison {  
        std::cmp::Ordering::Less => println!("Il primo vettore è minore"),  
        std::cmp::Ordering::Equal => println!("I due vettori sono uguali"),  
        std::cmp::Ordering::Greater => println!("Il primo vettore è maggiore"),  
    }  
}
```



eq(), ne(), lt(), le(), gt(), ge()

- 2 iteratori possono essere confrontati convertendoli prima in collezioni di valori comparabili e poi confrontando le collezioni utilizzando i metodi di confronto
- Il confronto avviene confrontando gli elementi corrispondenti delle 2 collezioni, considerando l'ordine degli elementi e il numero di elementi.

```
fn main() {  
    let numbers1 = vec![1, 2, 3];  
    let numbers2 = vec![4, 5, 6];  
  
    // Confronto dei due iteratori di numeri interi  
    let result = numbers1.iter().lt(numbers2.iter());  
  
    if result {  
        println!("Il primo iteratore è minore del secondo");  
    } else {  
        println!("Il primo iteratore non è minore del secondo");  
    }  
}
```



Esempio Finale

```
use itertools::Itertools;

fn main() {
    let nomi = vec!["alice", "bob", "charlie", "david", "eve", "carol"];
    let lettera_iniziale = 'c';
    let prefisso = "Dott. ";

    let nomi_trasformati: Vec<String> = nomi.into_iter()
        .filter(|nome| nome.starts_with(lettera_iniziale))
        .sorted_by_key(|&nome| nome.to_lowercase())
        .map(|nome| nome.to_uppercase())
        .map(|nome_maiuscolo| format!("{}", prefisso, nome_maiuscolo))
        .collect();

    println!("Nomi trasformati: {:?}", nomi_trasformati);
}
```



Potpourri

```
fn main() {  
    let sum: i32 = (1..=10).filter(|&x| x % 2 == 0).sum();  
    let reversed: String = "Rust!".chars().rev().collect();  
    let all_positive: bool = vec![1, 2, 3, 4].into_iter().all(|x| x > 0);  
    let max: Option<i32> = [5, 2, 8, 1, 9].into_iter().max();  
    let joined: String = vec!["Rust", "is", "awesome"].join(" ");  
  
    println!("sum: {:?}\nreversed: {:?}\nall_positive: {:?}\nmax: {:?}\njoined {:?}",  
            sum, reversed, all_positive, max, joined);  
}
```

Risorse utili

- The Rust Programming Language book - Chapter 13: "Functional Language Features: Iterators and Closures"
 - <https://doc.rust-lang.org/book/ch13-00-functional-features.html>
- Rust by Example - Iterators
 - <https://doc.rust-lang.org/rust-by-example/trait/iter.html>
- Rust Iterators: A Guide
 - <https://www.newline.co/@uint/rust-iterators-a-guide--80e35528>
- Daily Rust: Iterators
 - <https://adventures.michaelfbryan.com/posts/daily/iterators/>

Glossario

- **Iteratore**: struttura dati che evolve nel tempo, stateful, ossia mantiene uno stato interno che viene modificato durante l'iterazione. Lo stato interno determina quale sarà l'elemento successivo prodotto dall'iteratore.
- **Contenitore** (o collezione): struttura dati che contiene e gestisce un insieme di valori. I contenitori forniscono metodi per creare iteratori, consentendo di scorrere gli elementi in modo comodo ed efficiente
- **lazy evaluation** (pigro): un iteratore non esegue nessuna computazione o azione fino a quando non viene esplicitamente richiesto il suo prossimo elemento. In altre parole, la creazione di un iteratore non comporta immediatamente l'elaborazione di tutti gli elementi del contenitore. Le operazioni vengono rimandate fino al momento in cui è effettivamente necessario ottenere un valore. Solo a fronte di una richiesta che arriva dal lato destro (valle) l'iteratore fa qualcosa, se l'iteratore ha un dato lo fornisce, altrimenti propaga la richiesta a chi sta a sinistra (monte).
- **Eager evaluation** (ansiosa), in cui un'espressione viene valutata non appena viene incontrata, indipendentemente dal fatto che sia immediatamente necessario o meno.

Glossario

- **Iterable**: qualunque tipo che implementa il Tratto Intolterator
- **Valori** (values): dominio di dati prodotti da un iteratore
- **Item**: singolo valore prodotto da un iteratore
- **Adapter**: funzione che riceve gli items che un iteratore produce e genera un nuovo iteratore
- **Consumer**: funzione che consuma un iteratore e produce un risultato